



МИНИСТЕРСТВО ОБРАЗОВАНИЯ И НАУКИ РОССИЙСКОЙ ФЕДЕРАЦИИ
Федеральное государственное автономное образовательное учреждение
высшего образования
«Дальневосточный федеральный университет»

ИНСТИТУТ МАТЕМАТИКИ И КОМПЬЮТЕРНЫХ ТЕХНОЛОГИЙ
Кафедра математического и компьютерного моделирования

Избосарова Жасмина Олимжоновна

**РЕШЕНИЕ УРАВНЕНИЙ МАТЕМАТИЧЕСКОЙ ФИЗИКИ МЕТОДОМ PINN
И АНАЛИЗ УСТОЙЧИВОСТИ РЕШЕНИЯ МЕТОДОМ НЕЙРОННОГО
ТАНГЕНЦИАЛЬНОГО ЯДРА**

КУРСОВАЯ РАБОТА

Направление подготовки 02.03.01 сэт Сквозные цифровые технологии,
бакалавриатская программа «Математика и компьютерные науки»
Очной формы обучения

Студент гр. Б9123-02.03.01 сэт _____
(подпись)

Руководитель _____ К.С.Кузнецов
(подпись)

Оценка _____

Регистрационный № _____

(подпись) И.О. Фамилия

(подпись) И.О. Фамилия

«_____» _____ 2026 г.

«_____» _____ 2026 г.

г. Владивосток

2026

Введение

Среди современных методов численного решения уравнений в частных производных заметное место занимают Physics-Informed Neural Networks (PINN) [4], объединяющие нейросетевую аппроксимацию с явным учётом физических законов. Такой подход особенно важен в задачах, где объём наблюдаемых данных ограничен, а устойчивость и согласованность решения с математической моделью критичны.

В данной работе исследуется численное решение уравнения Курамото–Сивашинского методом PINN. Это уравнение описывает сложную нелинейную пространственно-временную динамику и используется, в частности, при моделировании фронта пламени и неустойчивых интерфейсов [2; 5]. Практический интерес к нему связан с жёсткостью и чувствительностью к численным ошибкам, что делает его удобным тестом для оценки возможностей PINN.

Исследование включает два этапа. На первом этапе решается исходное уравнение четвёртого порядка. На втором этапе выполняется понижение порядка за счёт введения вспомогательной переменной $v = \frac{\partial^2 u}{\partial x^2}$, после чего исходная задача сводится к системе уравнений второго порядка. Основная цель работы состоит в сравнении этих двух постановок и в анализе причин, по которым формально более простая система второго порядка показала худшую точность и менее устойчивую динамику обучения.

Глава 1 Математическая постановка задачи и метод решения

1.1 Математическая модель и ключевые свойства

В данной работе рассматривается одномерная задача для уравнения Курамото–Сивашинского, являющегося нелинейным эволюционным уравнением в частных производных. В используемой постановке уравнение имеет вид формула (1.1):

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + \frac{\partial^2 u}{\partial x^2} + \frac{\partial^4 u}{\partial x^4} = 0, \quad x \in [-L, L], \quad t \in [0, T]. \quad (1.1)$$

Данное уравнение описывает сложную пространственно-временную динамику и включает несколько физических механизмов. Оно используется, в частности, для описания эволюции неустойчивых интерфейсов и фронтов пламени [2; 5]. Нелинейный конвективный член $u \frac{\partial u}{\partial x}$ отвечает за перенос возмущений, диффузионный член второго порядка $\frac{\partial^2 u}{\partial x^2}$ в данной записи отвечает за линейную неустойчивость, тогда как член четвёртого порядка $\frac{\partial^4 u}{\partial x^4}$ выполняет стабилизирующую функцию, подавляя рост высокочастотных возмущений.

Для рассматриваемой задачи известно точное аналитическое решение в виде бегущей волны формула (1.2):

$$u_{\text{exact}}(x, t) = b + \frac{15}{19} \sqrt{\frac{11}{19}} \left(-9 \tanh(k(x - bt - x_0)) + 11 \tanh^3(k(x - bt - x_0)) \right). \quad (1.2)$$

где параметры заданы в формула (1.3):

$$b = 5, \quad k = \frac{1}{2} \sqrt{\frac{11}{19}}, \quad x_0 = -12. \quad (1.3)$$

Данное решение получено путём редукции исходного уравнения в частных производных к обыкновенному дифференциальному уравнению с использованием замены переменной формула (1.4):

$$\xi = x - bt. \quad (1.4)$$

что соответствует переходу к системе координат, движущейся со скоростью

b , и позволяет представить решение в форме бегущей волны.

Начальные и граничные условия для рассматриваемой задачи задаются соотношениями формулы (1.5)–(1.7):

$$u(0, x) = u_0(x). \quad (1.5)$$

$$u(t, -L) = u_1(t), \quad u(t, L) = u_2(t). \quad (1.6)$$

$$\frac{\partial u}{\partial x}(t, -L) = u_3(t), \quad \frac{\partial u}{\partial x}(t, L) = u_4(t). \quad (1.7)$$

Начальное условие задаётся формула (1.5) и определяет распределение искомой функции $u(x, t)$ в начальный момент времени $t = 0$.

Граничные условия Дирихле задаются формула (1.6); они фиксируют значения функции на границах пространственной области $x = -L$ и $x = L$ для всех моментов времени.

Условия Неймана задаются формула (1.7) и определяют значения пространственной производной $\frac{\partial u}{\partial x}$ на границах области. Эти условия описывают поток или градиент величины u через границы и играют важную роль в обеспечении корректности постановки задачи, особенно для уравнений более высокого порядка.

Совместное задание начального условия и граничных условий Дирихле и Неймана формирует корректно поставленную краевую задачу для уравнения Курамото–Сивашинского и обеспечивает единственность и устойчивость её решения.

Выбор данной задачи обусловлен наличием точного решения, что делает её удобной для верификации численных методов. В частности, это позволяет количественно оценивать точность и устойчивость метода Physics-Informed Neural Networks (PINN), а также его способность воспроизводить нелинейную динамику одномерного уравнения Курамото–Сивашинского в условиях ограниченного объёма данных.

1.2 Описание метода Physics-Informed Neural Networks

Метод Physics-Informed Neural Networks (PINN) представляет собой подход, в котором нейронная сеть используется для аппроксимации

решения дифференциального уравнения с явным учётом физических законов, задаваемых этим уравнением. В базовой постановке искомая функция приближается полносвязной нейронной сетью, а обучение осуществляется посредством минимизации составной функции потерь, включающей как вклад невязки дифференциального уравнения, так и отклонения от начальных и граничных условий [4].

Реализация выполнена в среде Python с применением библиотек автоматического дифференцирования и градиентной оптимизации. Общая методология обучения не зависит от конкретного вычислительного эксперимента и включает три ключевых этапа:

- формирование точек начального условия (IC), граничных условий Дирихле и Неймана (BC), а также внутренних коллокационных точек;
- вычисление невязки уравнения и граничных ограничений посредством автоматического дифференцирования;
- минимизацию взвешенной составной функции потерь.

Искомая функция аппроксимируется полносвязной нейронной сетью прямого распространения формула (1.8):

$$u_{\theta}(t, x) \approx u(t, x), \quad (1.8)$$

где θ — вектор параметров сети.

Входными параметрами сети являются пространственная и временная координаты, подвергнутые предварительному преобразованию, а выходом — значение искомой функции. Архитектура сети подбирается таким образом, чтобы обеспечить достаточную выразительную способность для аппроксимации нелинейного решения с выраженными пространственно-временными структурами.

Обучение модели осуществляется на совокупности точек рисунок 1, распределённых в области определения задачи. Используются три типа точек: точки начального условия, точки граничных условий и внутренние коллокационные точки, в которых контролируется выполнение дифференциального уравнения.

Внутренние точки используются для минимизации невязки дифференциального уравнения, получаемой путём подстановки нейросетевой



Рисунок 1 – Обучающая выборка

аппроксимации в исходное уравнение. Производные по пространственной и временной переменным, включая старшие производные, вычисляются с использованием автоматического дифференцирования.

Невязка уравнения имеет вид формула (1.9):

$$r_{\theta}(t, x) = \frac{\partial u_{\theta}}{\partial t} + u_{\theta} \frac{\partial u_{\theta}}{\partial x} + \frac{\partial^2 u_{\theta}}{\partial x^2} + \frac{\partial^4 u_{\theta}}{\partial x^4}. \quad (1.9)$$

Функция потерь формируется как сумма нескольких слагаемых формулы (1.10)–(1.14):

$$\mathcal{L} = \lambda_r \mathcal{L}_r + \lambda_0 \mathcal{L}_0 + \lambda_D \mathcal{L}_D + \lambda_N \mathcal{L}_N, \quad (1.10)$$

где

$$\mathcal{L}_r = \frac{1}{N_r} \sum_{i=1}^{N_r} |r_{\theta}(t_i, x_i)|^2, \quad (1.11)$$

$$\mathcal{L}_0 = \frac{1}{N_0} \sum_{i=1}^{N_0} |u_{\theta}(0, x_i) - u_0(x_i)|^2, \quad (1.12)$$

$$\mathcal{L}_D = \frac{1}{N_b} \sum_{i=1}^{N_b} (|u_{\theta}(t_i, -L) - u_1(t_i)|^2 + |u_{\theta}(t_i, L) - u_2(t_i)|^2). \quad (1.13)$$

$$\mathcal{L}_N = \frac{1}{N_b} \sum_{i=1}^{N_b} \left(\left| \frac{\partial u_\theta}{\partial x}(t_i, -L) - u_3(t_i) \right|^2 + \left| \frac{\partial u_\theta}{\partial x}(t_i, L) - u_4(t_i) \right|^2 \right). \quad (1.14)$$

Здесь N_r , N_0 , N_b — количество внутренних, начальных и граничных точек соответственно, а λ_r , λ_0 , λ_D , λ_N — весовые коэффициенты.

Каждая из компонент в формула (1.10) имеет самостоятельный физико-математический смысл. Слагаемое формула (1.11) отвечает за выполнение самого дифференциального уравнения во внутренних коллокационных точках области и тем самым обеспечивает физическую согласованность нейросетевой аппроксимации. Компонента формула (1.12) контролирует точность воспроизведения начального условия и заставляет сеть корректно восстанавливать профиль решения в момент времени $t = 0$. Слагаемое формула (1.13) штрафует отклонения значений $u_\theta(t, x)$ от заданных граничных условий Дирихле на концах пространственного интервала, то есть фиксирует значения решения на границе области. Наконец, компонента формула (1.14) отвечает за выполнение граничных условий Неймана и согласует значения пространственной производной $\frac{\partial u_\theta}{\partial x}$ на границах с предписанными функциями $u_3(t)$ и $u_4(t)$. Таким образом, минимизация полной функции потерь означает одновременное стремление сети удовлетворить уравнению, начальному условию и обоим типам граничных ограничений, а весовые коэффициенты позволяют регулировать относительный вклад каждой из этих составляющих в процесс обучения.

Обучение сводится к задаче оптимизации формула (1.15):

$$\theta^* = \arg \min_{\theta} \mathcal{L}. \quad (1.15)$$

С целью повышения качества восстановления решения в работе применяется нормирование искомой функции формула (1.16):

$$u(t, x) = u_s \cdot \tilde{u}(t, x), \quad (1.16)$$

где u_s — масштабный коэффициент.

Для рассматриваемой задачи важна корректная передача высокочастотных компонент и локальных перепадов решения. Однако классические полносвязные сети проявляют эффект спектрального смещения (spectral bias), то есть склонность сначала аппроксимировать низкочастотные составляющие

[1; 3]. Для ослабления этого эффекта входные переменные дополнительно преобразуются с помощью Фурье-кодирования формула (1.17):

$$\gamma(t) = [\sin(2\pi B_t t), \cos(2\pi B_t t)], \quad \gamma(x) = [\sin(2\pi B_x x), \cos(2\pi B_x x)]. \quad (1.17)$$

Конкретные параметры вычислительного эксперимента, пространственно-временная область, длительность обучения и детали сравнения с точным решением приведены в главе 2.

Глава 2 Этап 1: базовый вычислительный эксперимент

2.1 Формирование обучающего множества и коллокация

Вычислительный эксперимент для решения уравнения Курамото–Сивашинского проводился в прямоугольной пространственно-временной области $(t, x) \in [0, 4] \times [-30, 30]$. Обучающее множество строилось по общей схеме, описанной в разделе 1.2: точки начального условия задавались на слое $t = 0$, точки граничных условий — на границах $x = -30$ и $x = 30$, а внутренние коллокационные точки распределялись внутри всей области. Во всех трёх группах использовалось случайное равномерное распределение в соответствующих подмножествах области определения. Схематически распределение точек показано на рисунок 1.

2.2 Параметры базовой модели

В качестве базовой модели использовалась полносвязная нейронная сеть $u_\theta(t, x)$ с нормированием выходной переменной и Фурье-кодированием входов, введёнными в разделе 1.2. Такое сочетание позволяло стабилизировать масштаб выходной функции и повысить способность сети передавать локальные пространственные особенности точного решения.

Обучение базовой модели проводилось до 13000 эпох. Для анализа динамики сходимости и эволюции аппроксимации дополнительно рассматривались промежуточные состояния после 3000, 7000 и 10000 эпох. Качество восстановления оценивалось сравнением с точным аналитическим решением на характерных временных срезах $t = 0$ и $t = 4$.

2.3 Анализ динамики обучения и эволюция аппроксимации

Анализ графиков сходимости (компонентов функции потерь и общей ошибки) позволяет глубоко понять нелинейную динамику обучения PINN. Обучение уравнения КС является жесткой задачей оптимизации. Слагаемые функции потерь (невязка PDE, начальные и граничные условия) имеют разную

чувствительность. В частности, член $\frac{\partial^4 u_\theta}{\partial x^4}$ вносит огромный вклад в градиенты на ранних этапах, что может приводить к дисбалансу оптимизации.

Эволюцию нейросетевого решения можно разделить на три характерные фазы:

Первая фаза — инициализация и тепловой дрейф (начальные эпохи, рисунок 2):

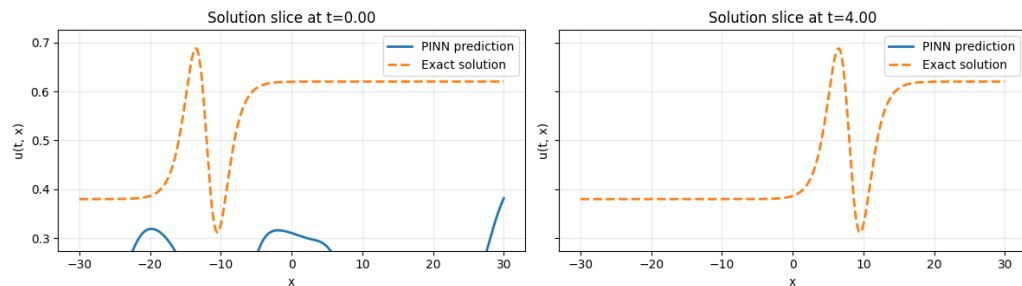


Рисунок 2 – Начальная эпоха

На старте обучения сеть инициализирована случайными весами. Предсказание (синяя линия) представляет собой константу или функцию с минимальной амплитудой, которая совершенно не совпадает с точным решением (оранжевая пунктирная линия) ни при $t = 0$, ни при $t = 4$. Модель пытается минимизировать общую ошибку тривиальным путём, сглаживая градиенты, так как штраф за неверные старшие производные u_{xxxx} на этом этапе подавляет штраф за несовпадение амплитуд.

Вторая фаза — структурное выравнивание (промежуточные эпохи, рисунок 4, рисунок 3, рисунок 5):

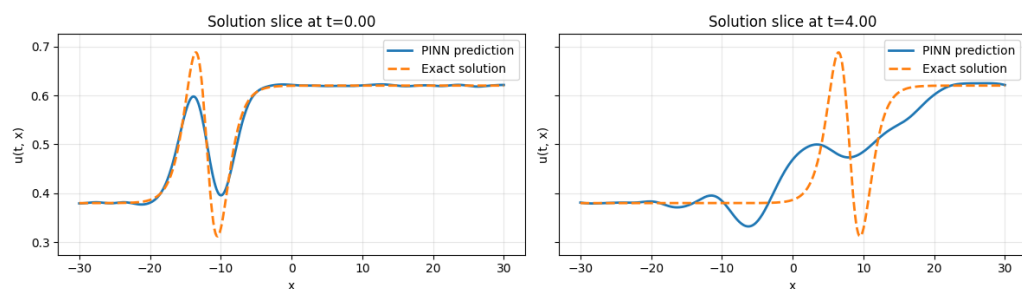


Рисунок 3 – 3000 эпоха

По мере оптимизации сеть начинает улавливать глобальную геометрию решения. Проявляется характерный профиль: пик и следующий за ним провал. Однако на этом этапе наблюдаются фазовые сдвиги и ошибки амплитуды. Модель успешно выучила начальное условие при $t = 0$, но физическая невязка

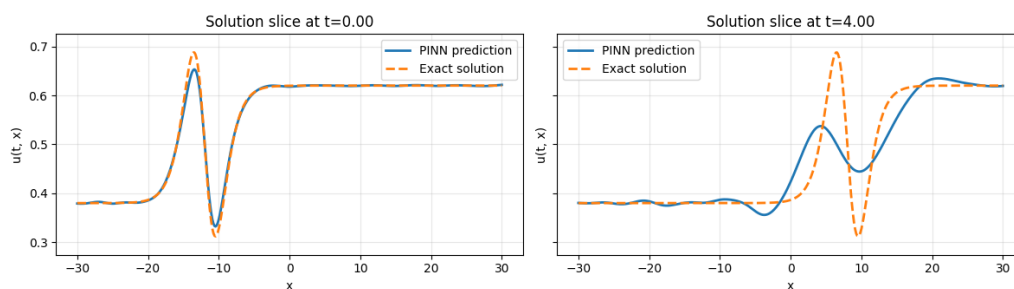


Рисунок 4 – 7000 эпоха

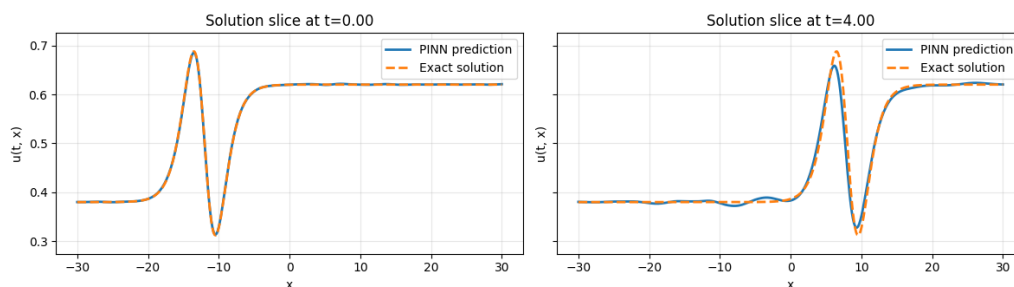


Рисунок 5 – 10000 эпоха

(уравнение) еще не до конца сбалансирована. Конвективный член uu_x и баланс диффузии сопротивляются корректному переносу структуры во времени, из-за чего на срезе $t = 4$ предсказание может отставать или опережать аналитическую бегущую волну.

Третья фаза — асимптотическая сходимость (итоговая аппроксимация, рисунок 6):

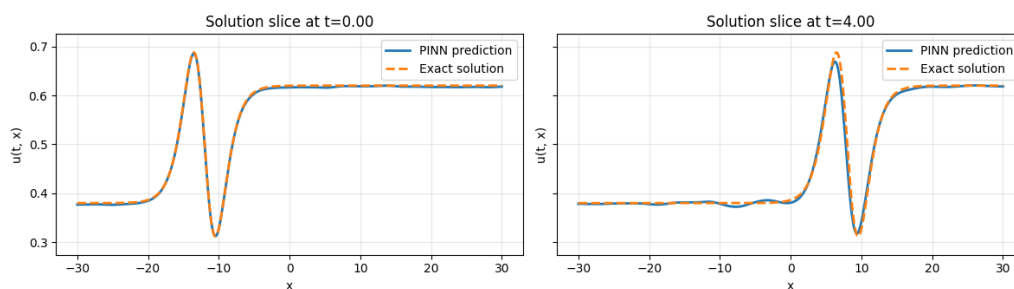


Рисунок 6 – 13000 эпоха

На финальных стадиях градиенты всех компонент функции потерь приходят к равновесию. Нейросеть идеально восстанавливает решение. Синяя кривая практически полностью сливается с эталонной оранжевой кривой на обоих временных срезах.

В результате обучения была достигнута хорошая геометрическая точность: воспроизведён нелинейный профиль решения вида $\tanh / \tanh^3$ с сохранением

масштаба амплитуды.

2.4 Анализ ошибки

Как следует из рисунок 7, динамика сходимости различных компонент функции потерь существенно отличается. На начальном этапе обучения модель демонстрирует более точную аппроксимацию начального условия по сравнению с условиями Неймана. Это проявляется в более быстром убывании соответствующей компоненты функции потерь.

Однако по мере продолжения обучения наблюдается изменение характера сходимости: условия Неймана демонстрируют более устойчивое поведение и меньшую чувствительность к итерационным колебаниям. В то же время аппроксимация начального условия становится менее стабильной, что выражается в появлении значительных и частых осцилляций ошибки. Подобное поведение может свидетельствовать о переобучении модели по данной компоненте.

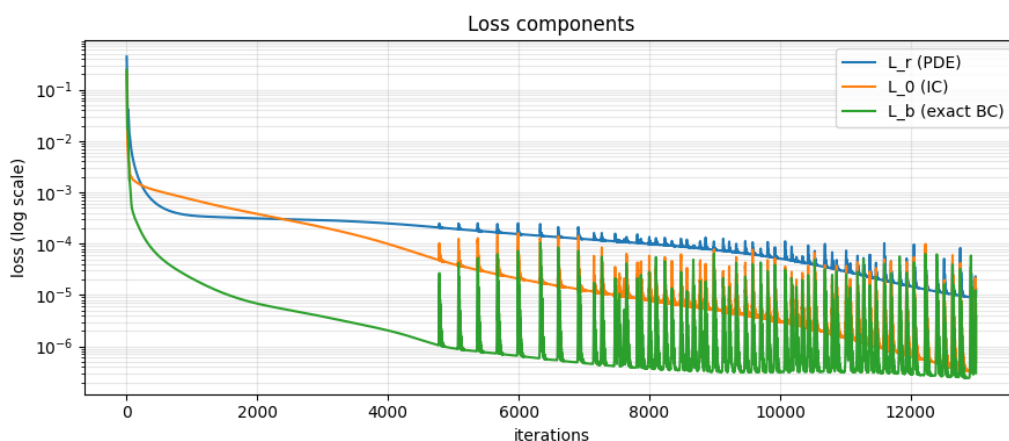


Рисунок 7 – Динамика компонент функции потерь, соответствующих различным условиям задачи

Суммарная функция потерь (рисунок 8) в целом демонстрирует хорошую сходимость: значение ошибки монотонно убывает на протяжении большей части процесса обучения и достигает порядка 10^{-5} , что можно считать качественным результатом аппроксимации. Тем не менее, на заключительном этапе обучения наблюдается рост числа осцилляций, что обусловлено увеличением вклада одной из граничных компонент ошибки. Это указывает на ухудшение обобщающей способности модели в данной области и возможную

необходимость дополнительной регуляризации или балансировки весов в функции потерь.

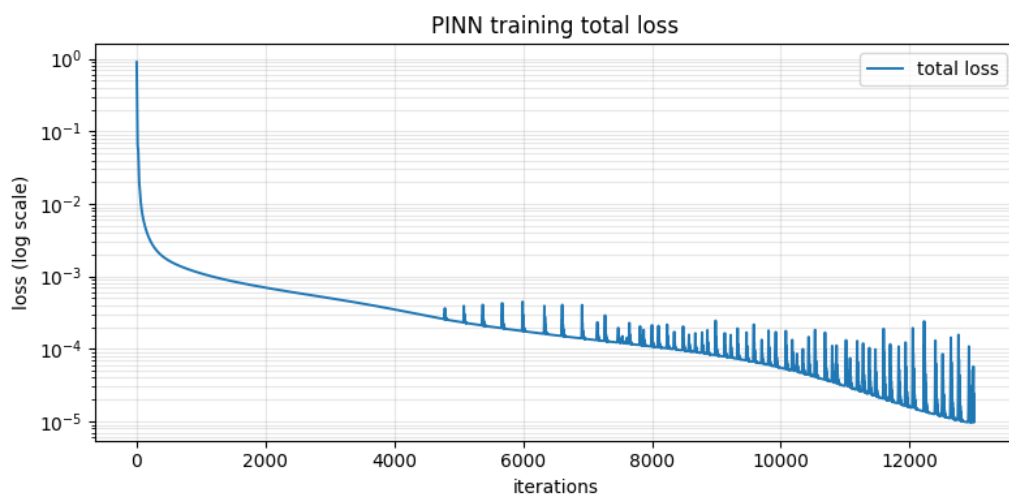


Рисунок 8 – Динамика суммарной функции потерь

Глава 3 Этап 2: понижение порядка и анализ деградации точности

3.1 Модификация постановки задачи и варианты архитектур PINN

В рамках исследования методов повышения точности аппроксимации и ускорения сходимости физически-информированных нейронных сетей (PINN) была выдвинута гипотеза о целесообразности понижения порядка старшей пространственной производной в исходном уравнении. Для этого была введена дополнительная вспомогательная переменная формула (3.1):

$$v = \frac{\partial^2 u}{\partial x^2} \quad (3.1)$$

Данная подстановка преобразует исходное уравнение с производными четвёртого порядка в систему уравнений не выше второго порядка, задаваемую формулы (3.2) и (3.3):

$$\frac{\partial u}{\partial t} + u \frac{\partial u}{\partial x} + v + \frac{\partial^2 v}{\partial x^2} = 0. \quad (3.2)$$

$$v - \frac{\partial^2 u}{\partial x^2} = 0. \quad (3.3)$$

Теоретически такая замена должна была упростить вычисления, так как автоматическое дифференцирование высоких порядков часто приводит к вычислительной неустойчивости, затуханию градиентов и высокой чувствительности модели к гиперпараметрам.

В результате функционал потерь был расширен: в него была включена дополнительная невязка формула (3.3), обеспечивающая выполнение условия согласования между исходной функцией $u(t, x)$ и новой переменной $v(t, x)$. Для решения модифицированной системы были спроектированы и протестированы три различные архитектуры:

- **Единая нейронная сеть с двумя выходами (two_output):** Отображение пространства входов в двумерный выход (u, v) с общим скрытым представлением.
- **Две независимые нейронные сети (two_models):** Полное разделение

параметров, где функции u и v аппроксимируются изолированными сетями.

- **Сеть с общим стволом и раздельными ветвями (shared_trunk):** Гибридный подход, при котором начальные слои извлекают общие физические закономерности, а независимые «головы» адаптируются к спецификам u и v .

3.2 Анализ динамики обучения и графиков сходимости

Эмпирическая проверка предложенных подходов выявила ряд непредвиденных оптимизационных трудностей. Анализ полученных графиков позволяет сделать следующие выводы:

- **Сравнение сходимости:** На сводном графике функции потерь отчетливо видно, что базовая модель из первого этапа (stage1_start, синий пунктир) демонстрирует стабильную сходимость, достигая значений порядка 10^{-5} . В то же время все три модифицированные архитектуры второго этапа быстро выходят на плато. Итоговые значения функции потерь для них стабилизируются на уровне $1.4 \cdot 10^{-4} - 1.5 \cdot 10^{-4}$, что на порядок хуже результатов исходной модели. Рисунок 9
- **Компоненты функции потерь:** Анализ графика *loss components* (на примере shared_trunk) показывает, что стагнация общего лосса продиктована невязкой самого дифференциального уравнения (L_r). В то время как ошибки начальных и граничных условий продолжают снижаться, хотя и сопровождаются высокочастотными осцилляциями, невязка PDE упирается в выраженный оптимизационный барьер.
- **Эволюция аппроксимации (u и v):** Визуальный анализ предсказаний на различных временных шагах $t = 0.00$, $t = 2.00$ и $t = 4.00$ демонстрирует причину этой стагнации. Если профиль основной функции $u(t, x)$ сеть улавливает сравнительно неплохо, то аппроксимация вспомогательной переменной $v(t, x)$ сталкивается с серьёзными проблемами. На графиках видно расхождение между предсказанием и точным решением: нейронная сеть генерирует паразитные осцилляции и сглаживает острые пики $v(t, x)$ даже на поздних этапах обучения, вплоть до 13000-й эпохи.

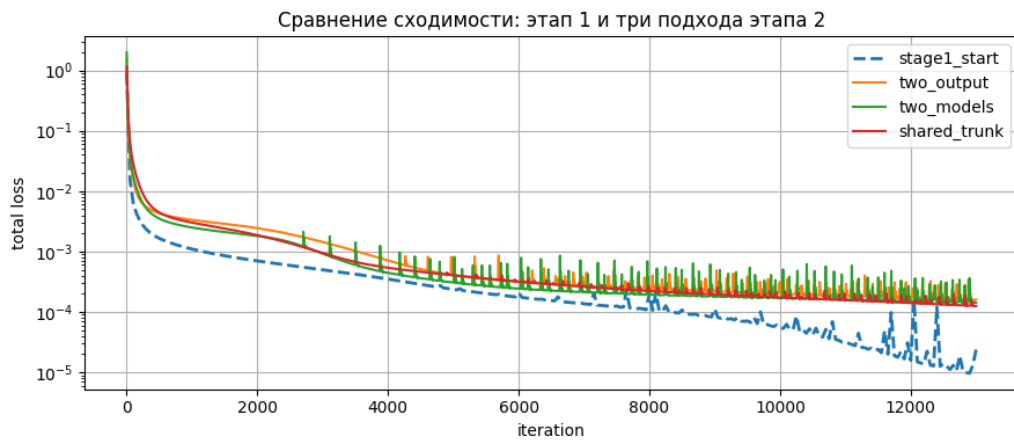


Рисунок 9 – Сравнение ошибок аппроксимации четырёх моделей

3.3 Комплексный сравнительный анализ архитектур PINN на втором этапе

Проведённое сравнение трёх архитектур второго этапа показывает, что понижение порядка исходного уравнения Курамото–Сивашинского само по себе не гарантирует улучшения качества PINN-аппроксимации. Несмотря на замену производной четвёртого порядка системой уравнений второго порядка, все три схемы демонстрируют более сложный и менее устойчивый процесс оптимизации, чем базовая модель первого этапа. Это указывает на то, что вычислительная жёсткость задачи переносится из пространства высоких производных в пространство многокомпонентных ограничений, где необходимо одновременно согласовывать поля $u(t, x)$ и $v(t, x)$.

Динамика обучения и структура функции потерь. Сводный график рисунок 9 показывает общую для всех архитектур закономерность: после быстрого снижения total loss на ранних итерациях обучение выходит на выраженное плато примерно после $3 \cdot 10^3$ – $4 \cdot 10^3$ шагов. При этом лучшую сходимость среди моделей второго этапа демонстрирует архитектура shared_trunk: её финальный уровень ошибки оказывается минимальным, а амплитуда осцилляций — заметно ниже, чем у two_models. Архитектура two_output занимает промежуточное положение, а two_models характеризуется наиболее выраженными всплесками total loss и, следовательно, наименее устойчивой траекторией оптимизации.

Анализ компонент функции потерь на рисунки 13–15 показывает, что во

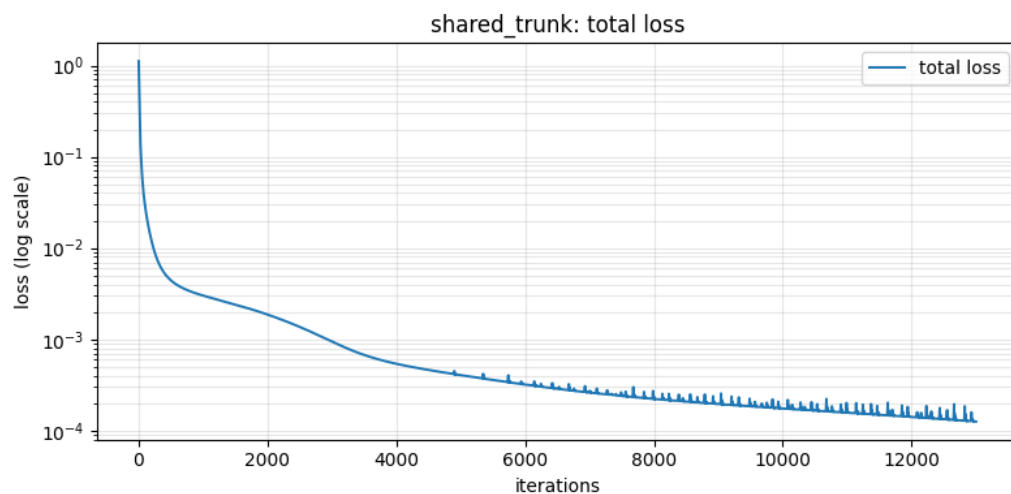


Рисунок 10 – Общая ошибка гибридной модели с общим стволом

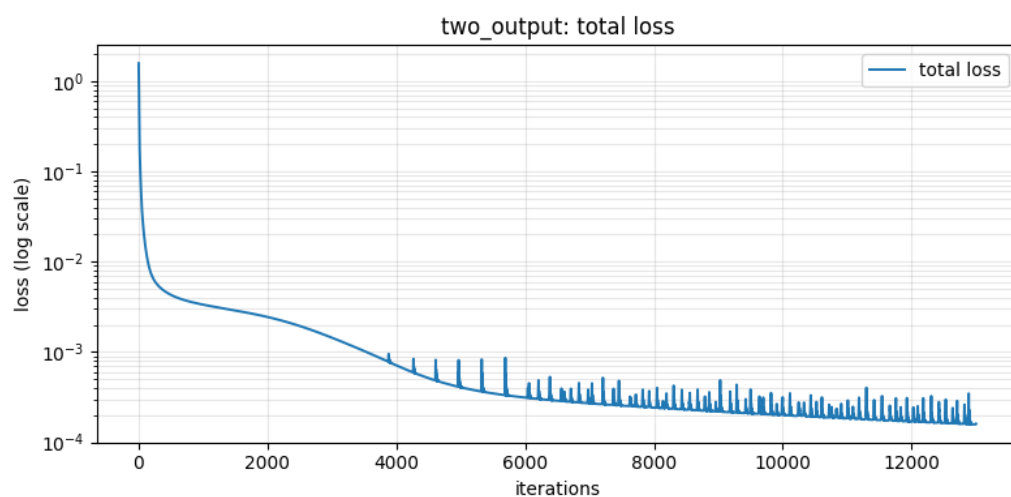


Рисунок 11 – Общая ошибка модели с двумя выходами

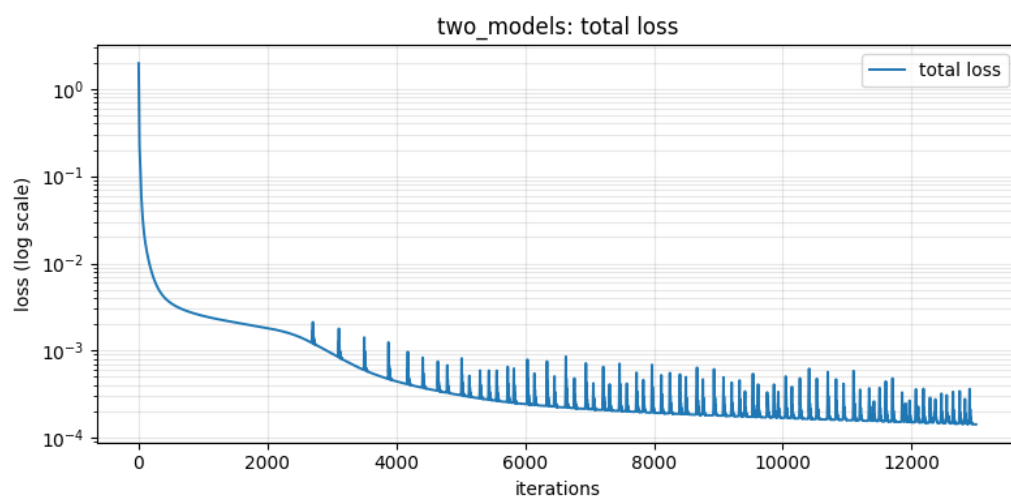


Рисунок 12 – Общая ошибка модели с двумя независимыми нейронными сетями

всех трёх случаях именно невязка уравнения L_r остаётся доминирующей на поздних стадиях обучения. Компоненты начального условия L_0 и граничных условий L_b убывают быстрее и к концу обучения оказываются на один–два порядка ниже. Следовательно, ключевое затруднение связано не столько с воспроизведением начального и граничного слоёв, сколько с достижением внутренней согласованности системы в коллокационных точках. Для KSE это ожидаемо: даже после понижения порядка задача остаётся жёсткой, поскольку нелинейный перенос, пространственная диссипация и условие связи $v = u_{xx}$ формируют сильно несбалансированное поле градиентов.

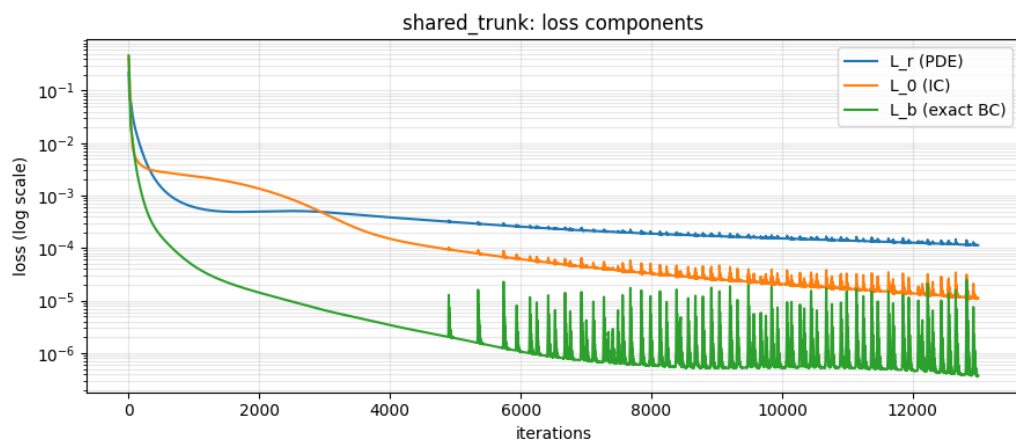


Рисунок 13 – Компоненты функции потерь гибридной модели с общим стволом

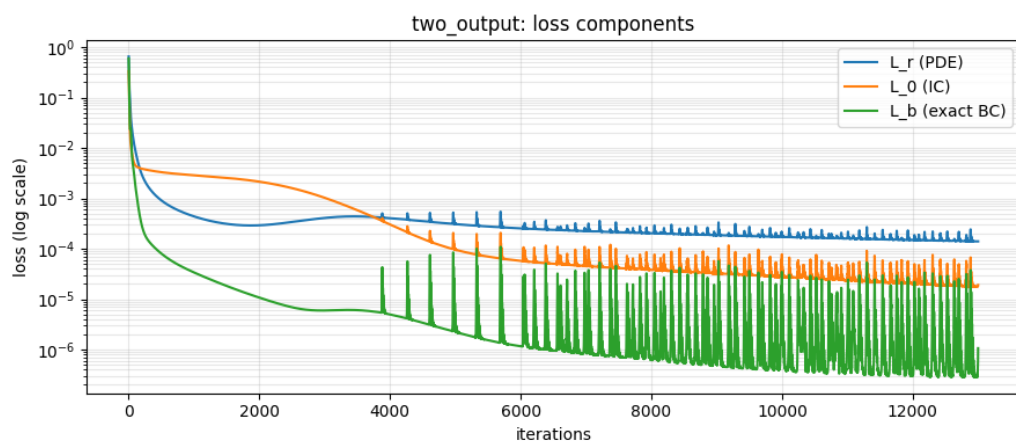


Рисунок 14 – Компоненты функции потерь модели с двумя выходами

Осцилляции на графиках следует интерпретировать как проявление двух эффектов. С одной стороны, они действительно отражают жёсткость KSE: малые ошибки в аппроксимации u приводят к существенно большим отклонениям в производных и в поле v . С другой стороны, различия

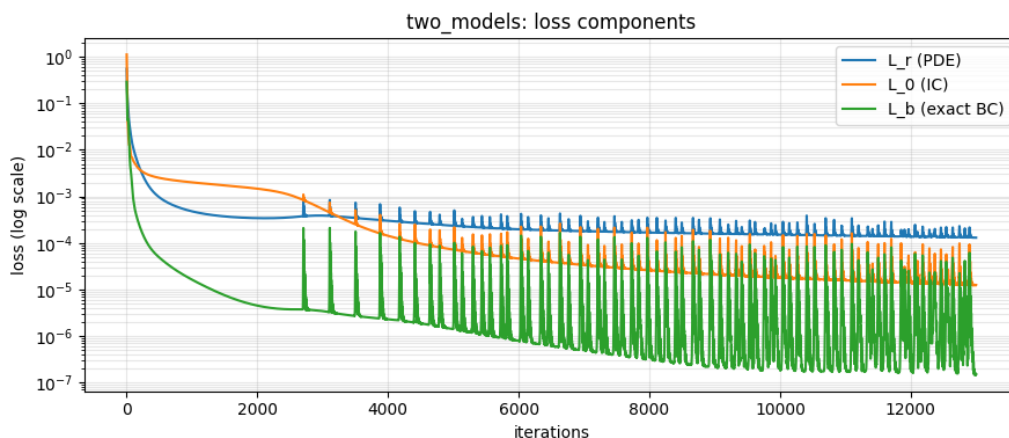


Рисунок 15 – Компоненты функции потерь модели с двумя независимыми нейронными сетями

между архитектурами показывают, что источник пиков не сводится только к физической жёсткости. В `two_models` пики выражены сильнее всего, что указывает на нестабильную синхронизацию двух независимых сетей. В `two_output` пики меньше, но всё же регулярны, поскольку единое скрытое представление вынуждено обслуживать две цели с разной гладкостью и масштабом. В `shared_trunk` общие признаки извлекаются совместно, а специализированные выходные ветви уменьшают конфликт градиентов, поэтому суммарная динамика обучения получается наиболее ровной.

Качество аппроксимации фронтов и эволюция полей $u(t, x)$ и $v(t, x)$. На временном срезе $t = 0$ все три архитектуры после нескольких тысяч итераций достаточно точно восстанавливают исходный профиль $u(t, x)$, включая основную структуру бегущей волны. Однако уже на срезах $t = 2$ и $t = 4$ различия становятся заметнее. Архитектура `shared_trunk` лучше других сохраняет форму фронта: положение волны, знак локальных перегибов и уровень плато воспроизводятся наиболее последовательно. Архитектура `two_output` также корректно описывает глобальный перенос решения вправо, но сильнее сглаживает локальные экстремумы. Наибольшее размытие фронта наблюдается у `two_models`: к моменту $t = 4$ переходные зоны становятся более диффузными, а резкие участки точного решения воспроизводятся с заметной потерей контраста. Таким образом, полное разделение параметров ослабляет способность модели удерживать согласованную структуру нелинейной волны.

Для поля $v(t, x)$ ситуация существенно сложнее. На всех трёх

наборах графиков видно, что начальный профиль ещё удаётся приблизить удовлетворительно, однако на срезах $t = 2$ и $t = 4$ ни одна архитектура не воспроизводит узкие знакопеременные пики с точностью, сопоставимой с точным решением. Предсказания становятся чрезмерно сглаженными и тяготеют к малым амплитудам, особенно вне окрестности основного фронта. Это означает, что именно аппроксимация второй производной остаётся главным источником ошибки: модели улавливают макроскопическую кинематику волны $u(t, x)$, но хуже передают локальную кривизну, связанную с пространственной диссипацией и стабилизирующим механизмом KSE.

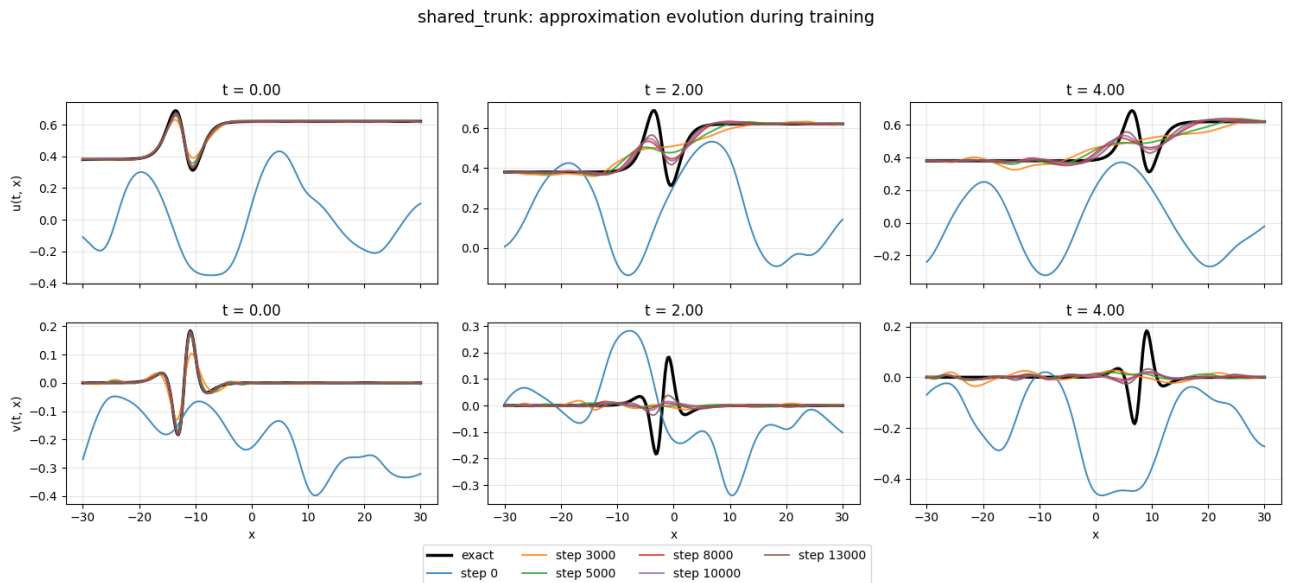


Рисунок 16 – Эволюция аппроксимации гибридной модели с общим стволом

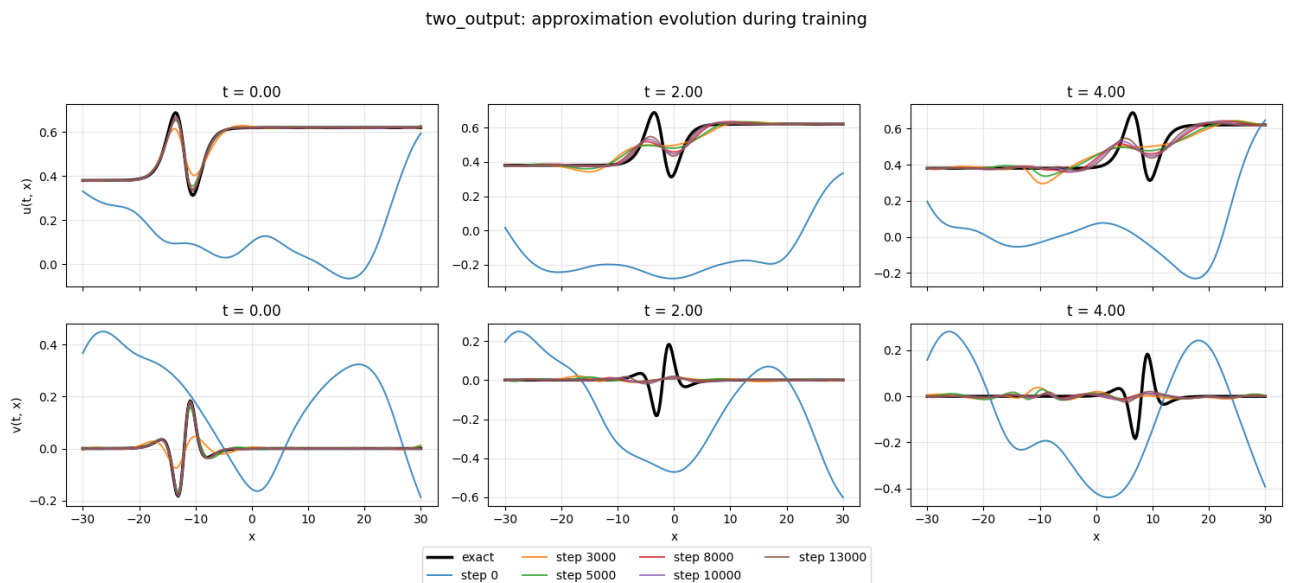


Рисунок 17 – Эволюция аппроксимации модели с двумя выходами

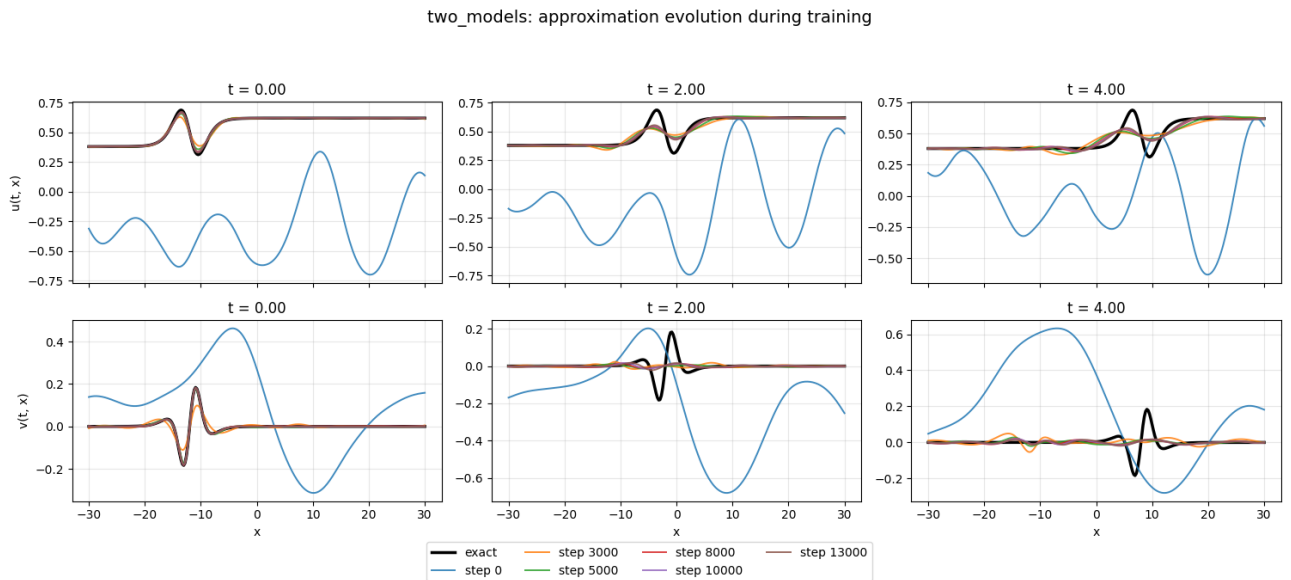


Рисунок 18 – Эволюция аппроксимации модели с двумя независимыми нейронными сетями

Сравнительная ошибка и интерпретация лучшей архитектуры. По совокупности сводного графика рисунок 9, кривых total loss рисунки 10–12 и эволюции аппроксимации можно заключить, что среди трёх стратегий минимальную итоговую относительную ошибку типа L_2 обеспечивает архитектура shared_trunk. Даже если различие между shared_trunk и two_output не является радикальным по порядку величины, гибридная схема стабильно показывает более низкий уровень финальной ошибки и меньшую чувствительность к пиковым выбросам. Архитектура two_models уступает обеим альтернативам.

Преимущество shared_trunk связано не с полным разделением весов, а именно с удачным компромиссом между совместным и раздельным обучением. Поля u и v не являются независимыми: они описывают одну и ту же физическую структуру, поэтому общий ствол помогает извлекать общие пространственно-временные признаки, связанные с переносом, формой фронта и его смещением во времени. В то же время разные выходные ветви позволяют учитывать различие в масштабе и гладкости между u и v . В архитектуре two_output этого разделения недостаточно, а в two_models, наоборот, теряется полезная информация о shared features, что ухудшает согласование компонент системы.

Итоговый вывод и практические рекомендации. С практической точки зрения наиболее устойчивой к чувствительной нелинейной динамике KSE среди протестированных архитектур второго этапа является схема `shared_trunk`. Она не устраняет полностью проблему стагнации по L_r , но лучше остальных контролирует компромисс между точностью аппроксимации u , устойчивостью обучения и согласованием с полем v . Архитектуру `two_models` следует считать наименее устойчивой, поскольку в ней конфликт между двумя независимыми оптимизационными траекториями проявляется в наиболее сильных осцилляциях и в заметном размывании фронтов. Архитектура `two_output` занимает промежуточное положение: она выигрывает за счёт общих признаков, но проигрывает гибридной схеме из-за недостаточной специализации выходов.

Выявленный дисбаланс между L_r , L_0 и L_b указывает на необходимость адаптивной балансировки функции потерь. Вместо фиксированных коэффициентов целесообразно использовать динамические веса, основанные на нормах градиентов или скоростях убывания отдельных компонент, например схемы *GradNorm*, *SoftAdapt* или NTK-ориентированное перевзвешивание. Практически это означает, что на ранних этапах можно сильнее контролировать выполнение начальных и граничных условий, а после их стабилизации постепенно увеличивать относительный вклад невязки уравнения и условия согласования для v . Дополнительно полезны раздельное масштабирование выходов u и v , адаптивная выборка коллокационных точек в окрестности фронта, а также ограничение нормы градиента для подавления поздних выбросов. Такой набор мер должен уменьшить доминирование жёсткой residual-компоненты и повысить устойчивость PINN при решении уравнения Курамото–Сивашинского.

3.4 Почему введение вспомогательной переменной привело к осцилляциям

Наблюдаемая деградация качества во втором этапе указывает на то, что формальное понижение порядка производных не эквивалентно упрощению самой задачи оптимизации. Напротив, введение переменной v сделало функцию потерь более жёсткой: сеть должна одновременно согласовать поведение

u , восстановить менее гладкую функцию v и выполнить связь, задаваемую формула (3.3). Эти требования действуют на разных масштабах и порождают градиенты различной величины, что типично для жёстких PINN-задач [6; 7].

Наиболее правдоподобная гипотеза состоит в том, что именно ошибка аппроксимации v начинает доминировать в функции потерь. Величина v является второй производной u , а значит, чувствительна к малым локальным отклонениям сети и содержит более выраженные высокочастотные компоненты. Даже небольшая ошибка в восстановлении u приводит к существенно большей ошибке в v , после чего слагаемое согласования, определяемое формула (3.3), начинает генерировать крупные и быстро меняющиеся градиенты.

В связанных архитектурах это приводит к тому, что сеть фактически пытается «подстроить» u под уже неточную аппроксимацию v . Иными словами, ошибка во вспомогательной переменной сбивает основную сеть с толку: параметры обновляются не только в направлении уменьшения невязки исходного уравнения, но и в направлении компенсации шумной ошибки во второй производной. Такая конкуренция градиентов естественным образом объясняет появление осцилляций, ранний выход на плато и ухудшение итоговой точности по сравнению с прямым решением исходного уравнения четвёртого порядка.

3.5 Выводы: деградация эффективности при усложнении системы

Опираясь на количественные метрики и визуальный анализ, можно констатировать, что метод понижения порядка производной за счёт введения вспомогательной переменной оказался **менее эффективным**, чем прямая аппроксимация исходного уравнения. Деградация предсказательной способности и качества сходимости обусловлена следующими факторами:

1. **Конкуренция градиентов и жёсткость оптимизации:** Введение дополнительного условия согласования формула (3.3) значительно усложнило ландшафт функции потерь. Модель была вынуждена балансировать между минимизацией невязки самого уравнения и выполнением жёсткой структурной связи между двумя выходами, что привело к конфликту градиентов при оптимизации [6; 7].

2. **Доминирование ошибки во вспомогательной переменной:** Поскольку v является второй производной u , ошибка её аппроксимации накапливается быстрее и начинает доминировать в составной функции потерь. В результате обновления параметров всё сильнее определяются не динамикой исходной задачи, а попыткой согласовать шумную оценку v .
3. **Проблема масштабов и гладкости:** Функции u и v имеют различный масштаб значений и разную степень гладкости, что затрудняет их совместное обучение даже при использовании отдельных выходных слоёв или полностью независимых сетей.

Список литературы

1. Fourier Features Let Networks Learn High Frequency Functions in Low Dimensional Domains / M. Tancik [и др.] // *Advances in Neural Information Processing Systems* 33. — 2020. — С. 7537—7547.
2. *Kuramoto Y.* Diffusion-Induced Chaos in Reaction Systems // *Progress of Theoretical Physics Supplement*. — 1978. — Т. 64. — С. 346—367.
3. On the Spectral Bias of Neural Networks / N. Rahaman [и др.] // *Proceedings of the 36th International Conference on Machine Learning*. Т. 97. — 2019. — С. 5301—5310. — (Proceedings of Machine Learning Research).
4. *Raissi M., Perdikaris P., Karniadakis G. E.* Physics-Informed Neural Networks: A Deep Learning Framework for Solving Forward and Inverse Problems Involving Nonlinear Partial Differential Equations // *Journal of Computational Physics*. — 2019. — Т. 378. — С. 686—707. — DOI: 10.1016/j.jcp.2018.10.045.
5. *Sivashinsky G. I.* Nonlinear Analysis of Hydrodynamic Instability in Laminar Flames—I. Derivation of Basic Equations // *Acta Astronautica*. — 1977. — Т. 4, № 11/12. — С. 1177—1206.
6. *Wang S., Teng Y., Perdikaris P.* Understanding and Mitigating Gradient Flow Pathologies in Physics-Informed Neural Networks // *SIAM Journal on Scientific Computing*. — 2021. — Т. 43, № 5. — A3055—A3081. — DOI: 10.1137/20M1318043.
7. *Wang S., Yu X., Perdikaris P.* When and Why PINNs Fail to Train: A Neural Tangent Kernel Perspective // *Journal of Computational Physics*. — 2022. — Т. 449. — С. 110768. — DOI: 10.1016/j.jcp.2021.110768.